

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE DIVISIÓN DE CIENCIAS DE LA
INGENIERÍA
INTRODUCCIÓN A LA PROGRAMACION Y COMPUTACION 2
PROYECTO 1
SEGUNDO SEMESTRE 2026
Fernan Andoni González Hernández 202331283

MANUAL TECNICO

EMPRESA: Anthropic

INSTITUCION: Ingeniería en ciencias y sistemas cunoc

FECHA: 17/08/2026

TECNOLOGIAS UTILIZADAS

LENGUAJE: Java 21

IDE: NetBeans 31

CONTROLADOR DE VERSIONES: GitHub

TIPO DE APLICACION: Consola

ENTRADA: archivos.pz

ESTRUCTURA DEL PROYECTO

Practical1Lenguajes

- Presenta el menu principal y las opciones que contenemos
- Solicita la ruta del archivo
- Valida que el archivo tenga extencion .pz
- Verifica que el archivo exista
- Lee el contenido
- Envia el contenido al analizador
- Muestra lo que son los tokens y los errores
- Genera lo que son los reportes

ANALIZADOR LEXICO

- Recibe el contenido del archivo
- Recorre las entradas caracter por caracter
- Identifica los diferentes tipos de tokens
- Registra los errores lexicos
- Continúa analizando despues de encontrar errores .

TOKEN

REPRESENTA LO QUE ES UN TOKEN RECONOCIDO

REPORTES

GENERA LOS REPORTES DE TOKENS Y ERRORES .

LISTA DE TOKENS DEL LENGUAJE

DIRECTIVA,
RESERVADA,
COMANDO_IA,
FUNCION,
CONECTOR,
IDENTIFICADOR,
LITERAL_CADENA,
LITERAL_ENTERO,
LITERAL_DECIMAL,
OPERADOR_ASIGNACION,
OPERADOR_CONCATENACION,
DELIMITADOR

TIPO DE TOKEN	DESCRIPCION	LEXEMA RECONOCIDO
DIRECTIVA	Representa lo que es una directiva que comienza con el @ y permite establecer configuraciones o características del lenguaje	@model @format @rol
RESERVADA	Palabras propias del lenguaje que tienen un significado específico y están reservadas para determinar instrucciones	AGENTE, contexto, variable, EJECUTAR, EXPORTAR.
COMANDO_IA	Representa comandos destinados a las	PREGUNTAR, GENERAR, RESUMIR,

	operaciones que realiza la inteligencia artificial	ANALIZAR, TRADUCIR, CLASIFICAR, EXTRAER.
FUNCION	Representa una funcion definida por el lenguaje para realizar una operacion.	CARGAR
CONECTOR	Representa palabras o simboos utilizados para establecer relaciones entre elementos.	SOBRE DESDE, EN, COMO, ->
IDENTIFICADOR	Representa lo que son nombres definidos por el usuario. Esta formado por letras, guion bajo y digitos.	nombre, archivo1, _dato
LITERAL_CADENA	Representa una secuencia de caracteres encerrada entre comillas dobles	"hola mundo" "hola" "realizar pregunta"
LITERAL_ENTERO	Representa un valor numerico unicamente formado por digitos.	10, 35, 2026
LITERAL_DECIMAL	Representa lo que es un valor numerico que tieene una parte separada mediante un punto.	10.5, 9.3, 10.43
OPERADOR_ASIGNACION	Representa el	=

	operador utilizado para poder asignar un valor	
OPERADOR_CONCATENACION	Representa el operador utilizado para concatenar elementos	+
DELIMITADOR	Representa caracteres utilizados para delimitar o separar diferentes elementos de una instruccion	{, }, (,),

EL CONECTOR puede estar representado tanto por determinadas palabras reservadas para conectar elementos (SOBRE, DESDE, EN COMO) como por el operador ->

RECORRIDO DE LA ENTRADA

Al momento de que el contenido del archivo .pz es recibido por el constructor **public AnalizadorLexico(String entrada)**

se inicializan lo que son las variables utilizadas para recorrer la entrada

Entrada: contiene todo el contenido del archivo.

Posicion: indica la posicion actual dentro de la entrada

Fila: indica la linea actual.

Columna: indica la columna actual

numeroToken: permite asignar un numero consecutivo a cada token

Recorrido caracter por caracter

El analisis se realiza mediante el metodo siguiente

public void analizar() el cual utiliza un ciclo **while** (**hayCaracteres()**) para poder contitnuar mientras existan caracteres por analizar, en cada iteracion se obtiene el caracter actual mediante **char actual = caracterActual();** en la cual el metodo **caracterActual** obtiene el caracter ubicado en ese posicion actual **return entrada.charAt(posicion);** de esta manera entonces el analizador procesa la entrada caracter por caracter.

AVANCE EN LA ENTRADA

Despues de procesar un caracter, el metodo **avanzar()** incrementa la posicion. Cuando el caracter es un salto de linea **\n** se incrementa la fila y la columna devuelve a 1 por lo que para los demas caracteres se incrementa unicamente la columna. Esto permite conocer lo que es la ubicacion exacta de los tokens y errores dentro del archivo.

ELEMENTOS IGNORADOS

ANTES DE RECONOCER LOS TOKESN EL ANALIZADOR VERIFICA SI EL CARACTER ACTUAL CORRESPONDE A UN ESPACIO. TABULACION, SALTO DE LINEA O RETORNO. LO CUAL LOS CARACTERES SON IGNORADOS MEDIANTE

```
if (esEspacio(actual)) {  
    avanzar();  
    continue;  
}
```

Tambien reconocen los comentarios de linea y comentarios de bloque. Lo cual son ignorados por el analizador.

FINALIZACION

Este proceso continua de manera que **posicion >= entrada.length()** cuando ya no existan caracteres. Termina el analisis y quedan almacenados los tokens reconocidos y los errores encontrados.

RECONOCIMIENTO DE TOKENS

Directivas

Cuando el caracter actual es @ el analizador llama a lo que es leerDirectiva() y en este metodo se obtiene el simbolo @ y continua leyendo letras _ y digitos hasta finalizar lo que es la directiva.

Por lo cual se verifica si la directiva pertenece a las reconocidas de nuestro lenguaje las cuales son

@modelo

@rol

@formato

Luego contenemos lo que es el leer numero cuando el caracter actual es un digito el metodo continua leyendo lo que son digitos consecutivos y si encuentra un punto lo incorpora al lexema y continua leyendo los siguientes digitos por lo cual clasificarNumero() determina si corresponde a LITERAL_NUMERO o LITERAL_DECIMAL

Y luego seguimos con lo que es las cadenas lo cual su metodo se llama leerCadenas() donde el analizador continua avanzando hasta encontrar otra comilla doble como "un hola mundo" se reconoce como LITERAL_CADENA. Lo cual si encuentra lo que es un salto de linea o se llega al final del archivo si encontrar la segunda comilla se genera el error cadena sin cerrar.

OPERADORES DE ASIGNACION

Cuando encuentra = se genera directamente lo que es el token de OPERADOR_ASIGNACION

Cuando encuentra - el analizador verifica que el siguiente caracter sea > si es asi entonces genera lo que es un token CONECTOR si despues de - no aparece > entonces se queda como un error por lo cual es un operador o conector no reconocido.

Cuando se encuentra con + se genera lo que es un token de OPERADOR_CONCATENACION

Ahora bien el metodo de esDeLimitaror() reconoce lo que son

```
{  
}  
(  
),  
,
```

Cada uno de estos caracteres genera un token DELIMITADOR

Ahora bien cuando el caracter actual es una letra o _ el analizador llama a leerIdentificador() este metodo entonces continua leyendo mientras encuentra letras, _ o digitos una vez obtenido el lexema utiliza clasificarPalabra() para determinar lo que es su tipo

Por ejemplo

AGENTE

Contexto

Variable

EJECUTAR

EXPLORAR

Son del tipo RESERVADA

Y las de

PREGUNTAR

GENERAR

RESUMIR

ANALIZAR

TRADUCIR

CLASIFICAR

EXTRAER

Sin del tipo COMANDO_IA